

RESEARCH PROJECT REPORT

Keyword Spotting on GAP9 and MAX78000FTHR

Embedded Keyword Spotting with Low-Power Neural Inference

Author: Vikash Kalyani Sankararaman
Email: vkalyani@student.ethz.ch
Group: 12
Date: 5 January 2025

Abstract

Keyword spotting (KWS) is an important always-on edge-AI workload in which a small neural network must recognize spoken commands under strict constraints on memory, computation, latency, and energy. This project investigates a 20-keyword KWS task based on the Google Speech Commands v2 dataset and evaluates two embedded platforms: the GAP9 system-on-chip and the MAX78000FTHR.

On GAP9, a baseline depthwise-separable convolutional neural network (DS-CNN) was first implemented to establish a compute-efficient reference. A customized DS-CNN was then developed by increasing feature-extraction capacity and network depth. After quantization-aware training (QAT), the baseline and customized models achieved 90.78% and 92.50% accuracy, respectively. Their measured inference times using the NE16 accelerator were approximately 0.774 ms and 4.99 ms.

On the MAX78000FTHR, several model-development strategies were explored. A larger hybrid teacher model was used for knowledge distillation, producing a compact student model with 126,580 parameters. The student achieved 89.084% accuracy after QAT, a model size of approximately 123 KB, 13.8 M MACC operations, and a measured inference time of 2.013 ms. A neural-architecture-search (NAS) model was also evaluated, achieving 88.76% accuracy with 426,076 parameters.

Among the deployed models reported in the project, the knowledge-distilled MAX78000 model provides the most favorable overall balance of accuracy, model size, and measured inference latency. The results demonstrate the trade-off between model capacity, accelerator utilization, memory footprint, and accuracy in embedded KWS.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Project Objectives	1
1.3	Report Structure	1
2	Problem Statement and Dataset	2
2.1	Problem Statement	2
2.2	Google Speech Commands v2	2
2.3	Feature Representation	2
3	Hardware Platforms	5
3.1	MAX78000FTHR and GAP9	5
3.2	Hardware-Aware Design Considerations	5
4	Keyword Spotting on GAP9	7
4.1	Baseline DS-CNN	7
4.2	Customized DS-CNN	8
4.3	GAP9 Results	8
4.4	Discussion	10
5	Keyword Spotting on MAX78000FTHR	11
5.1	Experimental Strategy	11
5.2	Teacher Model	11
5.3	Knowledge Distillation	11
5.3.1	Distillation Objective	13
5.4	Knowledge-Distilled Student Results	13
5.5	Energy and Throughput Estimates	13
5.6	Neural Architecture Search Model	15
6	Comparative Analysis	17

6.1	Overall Model Comparison	17
6.2	Accuracy, Size, and Latency Trade-offs	18
6.3	Selected Deployment Model	18
7	Conclusion	19
8	Future Work	20
8.1	Knowledge Distillation on GAP9	20
8.2	Improved Distillation Strategies	20
	References	21
	Source and Scope Note	22

List of Figures

2.1	Representative audio examples from the project dataset.	3
2.2	Class distribution reported for the keyword-spotting dataset.	3
2.3	Example MFCC representations used as input for the GAP9 DS-CNN pipeline. .	4
3.1	Hardware platforms considered in the project: MAX78000FTHR and GAP9. . .	5
3.2	GAP9 system architecture represented in the project material.	6
4.1	Baseline DS-CNN architecture used for GAP9.	7
4.2	Customized DS-CNN architecture for GAP9.	8
4.3	Baseline DS-CNN evaluation results on GAP9.	9
4.4	GAP9 performance summaries reported by the implementation.	9
4.5	Customized DS-CNN evaluation results on GAP9.	10
5.1	Teacher-model training and model-summary information reported in the project.	12
5.2	Knowledge-distillation training concept represented in the project material. . . .	12
5.3	Evaluation results for the knowledge-distilled student.	14
5.4	Training and QAT behavior of the MAX78000 student model.	14
5.5	NAS model training and model statistics.	15
5.6	Evaluation results for the NAS model.	16
6.1	Overall model comparison reproduced from the project evaluation.	17

List of Tables

3.1	Hardware comparison based on the project presentation.	6
4.1	GAP9 model comparison.	9
5.1	Knowledge-distilled student model on MAX78000FTHR.	13
5.2	NAS model performance reported for MAX78000FTHR.	15
6.1	Overall comparison of the reported KWS models.	17

Chapter 1

Introduction

1.1 Motivation

Keyword spotting is the task of detecting a small vocabulary of spoken commands from an audio stream. For embedded and always-on applications, the inference engine must operate with very low latency and energy consumption while fitting within a constrained memory budget. These requirements make KWS a representative workload for evaluating hardware-aware neural-network design.

This project targets a KWS20 system with 20 keywords. Compared with smaller keyword sets, the larger number of classes increases the classification difficulty, particularly when commands have similar phonetic characteristics. The project therefore focuses not only on recognition accuracy but also on deployment efficiency on two different embedded AI platforms.

1.2 Project Objectives

The project has four main objectives:

1. Develop a KWS20 system suitable for low-power, always-on inference.
2. Investigate lightweight neural architectures under strict memory, compute, and latency constraints.
3. Implement and evaluate keyword-spotting models on GAP9 and MAX78000FTHR.
4. Compare the final models using accuracy, parameter count, memory footprint, computational cost, and inference time.

1.3 Report Structure

The remainder of this report is organized as follows. Chapter 2 describes the problem formulation and dataset. Chapter 3 introduces the two target platforms. Chapter 4 presents the GAP9 implementation and experiments. Chapter 5 presents the MAX78000FTHR experiments, including knowledge distillation and neural architecture search. Chapter 6 summarizes the quantitative comparison, followed by conclusions and future work.

Chapter 2

Problem Statement and Dataset

2.1 Problem Statement

The target is a KWS20 system designed for always-on, low-power inference. The system must operate under strict constraints on:

- memory footprint,
- computational complexity,
- inference latency, and
- energy consumption.

The 20-keyword classification problem is more challenging than a smaller KWS task because the classifier must distinguish a larger number of classes, including acoustically or phonetically similar commands. The neural architecture must therefore provide sufficient representational capacity without becoming too expensive for embedded deployment.

2.2 Google Speech Commands v2

The project uses Google Speech Commands v2 (GSC v2). The audio is sampled at 16 kHz using 16-bit PCM representation, with approximately one second of audio per utterance. The dataset contains approximately 105,000 labeled utterances spoken by thousands of speakers, providing speaker diversity.

The project uses a KWS20 label set, together with explicit handling of silence and unknown examples during training. Fixed-length signals and standardized labels make the dataset suitable for embedded keyword-spotting benchmarking.

2.3 Feature Representation

For the GAP9 implementation, the processing pipeline is based on MFCC feature extraction followed by DS-CNN inference and multi-class classification. The MFCC representation provides a compact time-frequency representation that is suitable for embedded inference.

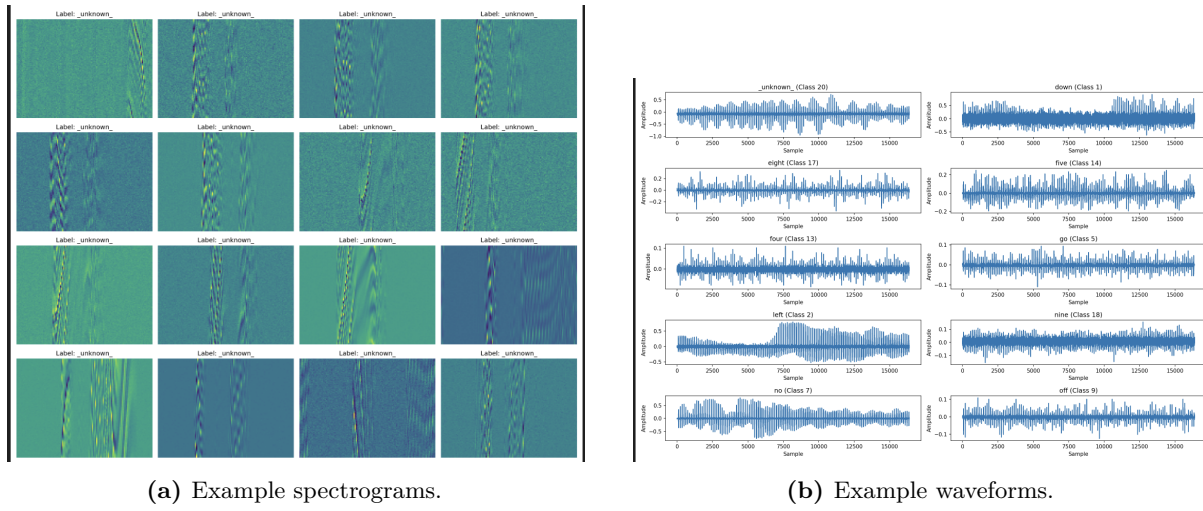


Figure 2.1: Representative audio examples from the project dataset.

Class Code	Word	Number of Utterances
0	up	1427
1	down	3511
2	left	3389
3	right	3592
4	stop	3322
5	go	3478
6	yes	3634
7	no	3343
8	on	3491
9	off	3449
10	one	3382
11	two	3298
12	three	1566
13	four	3328
14	five	3607
15	six	3461
16	seven	1810
17	eight	3379
18	nine	3536
20	_unknown_	32916

Figure 2.2: Class distribution reported for the keyword-spotting dataset.

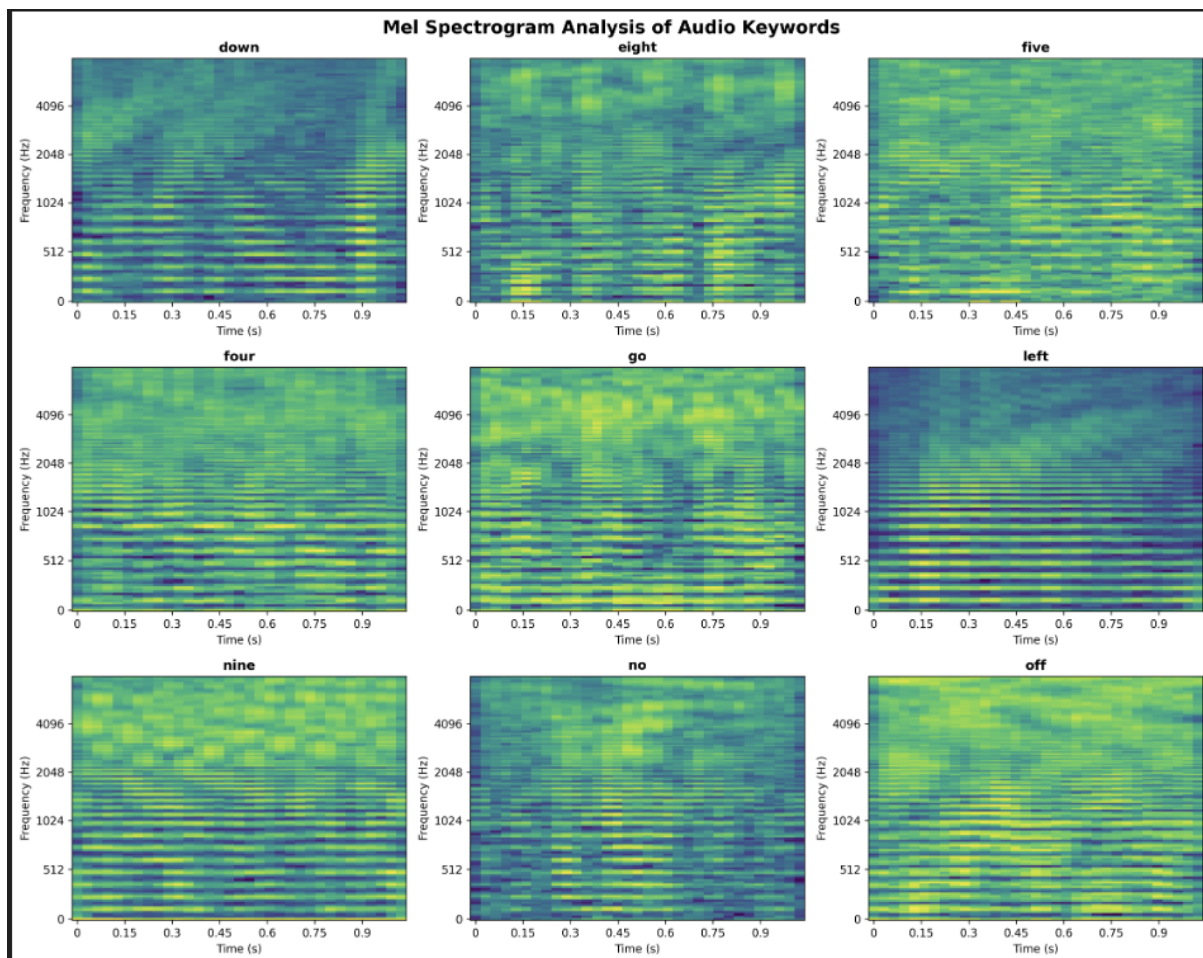


Figure 2.3: Example MFCC representations used as input for the GAP9 DS-CNN pipeline.

Chapter 3

Hardware Platforms

3.1 MAX78000FTHR and GAP9

Two platforms were considered because they provide different approaches to accelerating neural-network inference. The MAX78000FTHR is accelerator-centric, with a dedicated CNN accelerator and limited CPU involvement during inference. GAP9 combines a RISC-V compute cluster with the NE16 CNN accelerator and provides a hierarchical memory system.

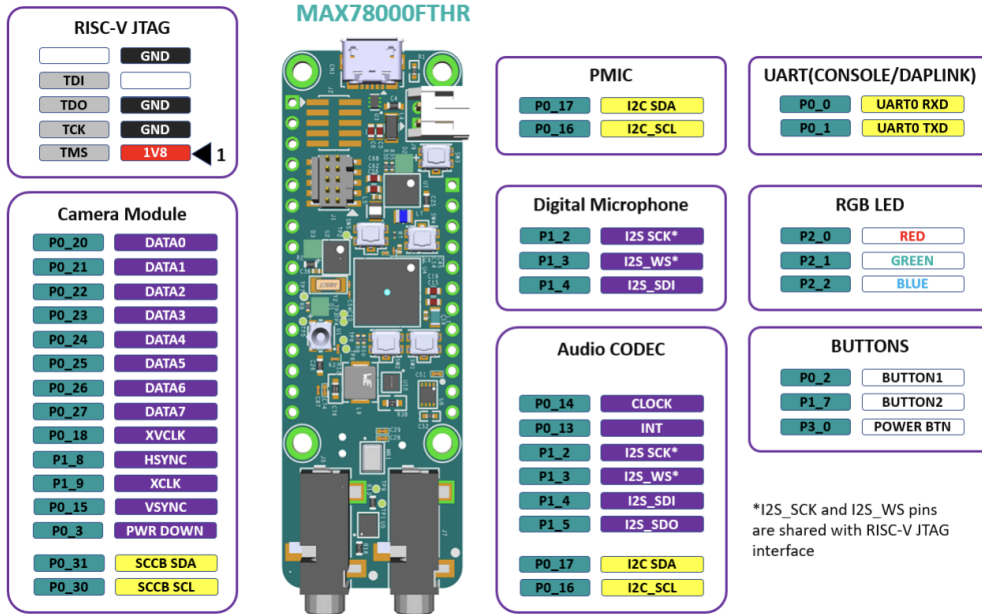


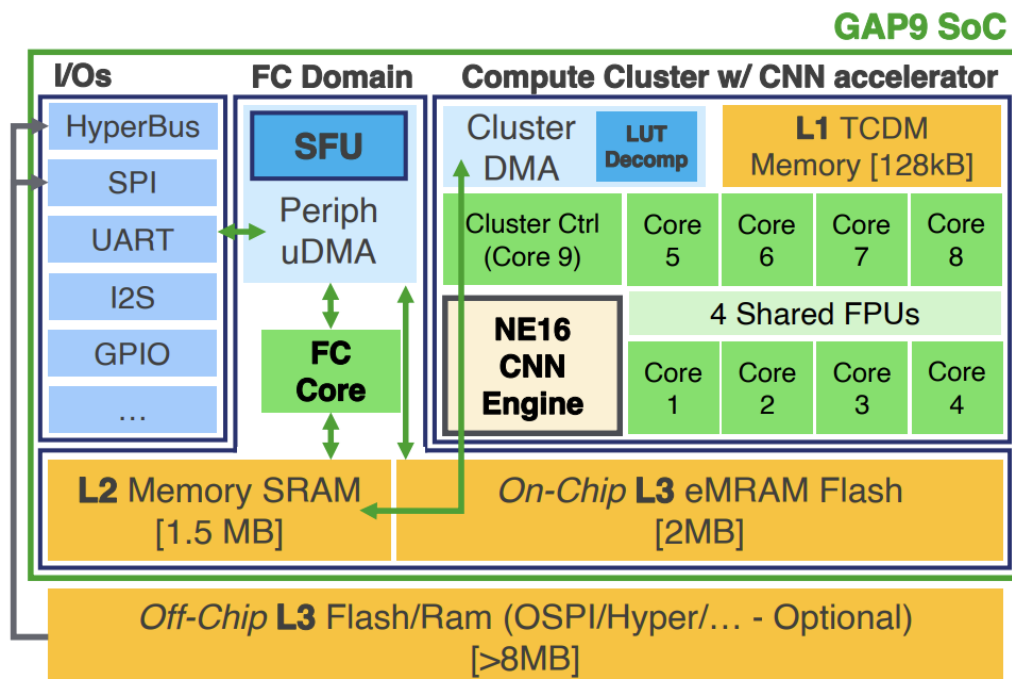
Figure 3.1: Hardware platforms considered in the project: MAX78000FTHR and GAP9.

3.2 Hardware-Aware Design Considerations

The two platforms lead to different design trade-offs. MAX78000FTHR emphasizes low-power, accelerator-driven CNN execution and is particularly well suited to always-on inference. GAP9 provides a more flexible execution environment, including explicit DMA-driven data movement, a multi-core compute cluster, and the NE16 accelerator. Consequently, GAP9 allows greater architectural flexibility at the cost of a more involved execution and memory-management model.

Table 3.1: Hardware comparison based on the project presentation.

Feature	MAX78000FTHR	GAP9
Processing architecture	Accelerator-centric; CPU as controller	Cluster-centric; CPU and accelerator orchestration
Main processing cores	Dual Arm Cortex-M4F + RISC-V control	1 FC + 8-core RISC-V compute cluster
Maximum clock frequency	≤ 100 MHz	≤ 370 MHz
CNN acceleration	Dedicated always-on CNN accelerator	NE16 CNN accelerator
CNN execution model	Fully hardware-driven, static graph	Tiled execution via NE16 + cluster
Parallelism	Fixed-function MAC array	NE16 + multi-core parallelism
Quantization	Native INT8 hardware	INT8 via NE16/software-assisted execution
On-chip memory	CNN SRAM + 128 KB system SRAM	L1 (128 KB) + L2 (1.5 MB)
Memory hierarchy	Flat, accelerator-local	Hierarchical (L1/L2/L3)
Data movement	Implicit, zero-copy	DMA-driven, explicit tiling
CPU involvement	Minimal	Moderate
Inference latency	Fully deterministic	Semi-deterministic
Always-on suitability	Excellent	Limited / duty-cycled
Model flexibility	CNN-focused	CNN, DSP, and hybrid workloads

**Figure 3.2:** GAP9 system architecture represented in the project material.

Chapter 4

Keyword Spotting on GAP9

4.1 Baseline DS-CNN

A baseline DS-CNN was implemented first as a compute-efficient reference model. The objective was to validate the complete pipeline:

Audio $\rightarrow$ MFCC extraction $\rightarrow$ DS-CNN inference $\rightarrow$ KWS classification.

The baseline provided a reference against which a higher-capacity architecture could be evaluated.

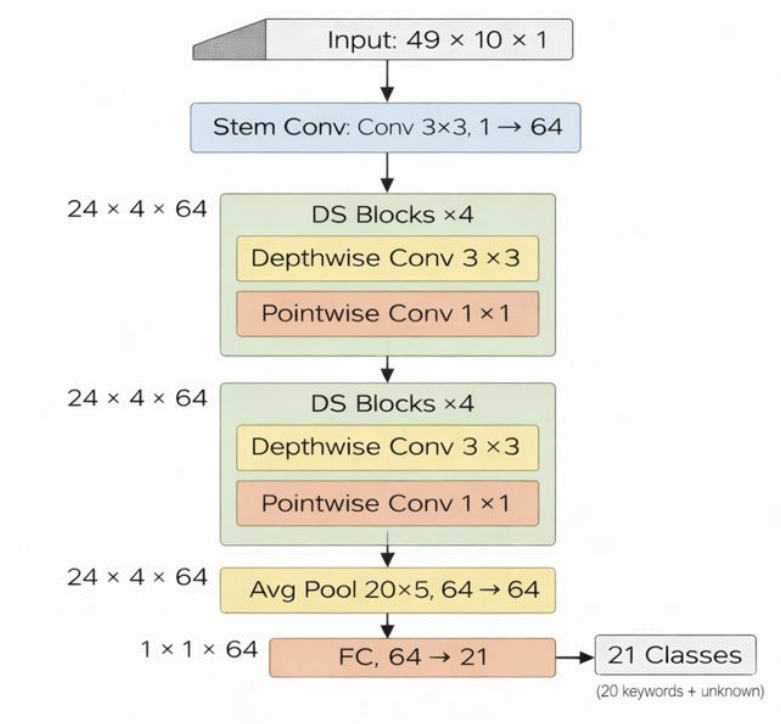


Figure 4.1: Baseline DS-CNN architecture used for GAP9.

4.2 Customized DS-CNN

The baseline model was not sufficiently strong for the more difficult 20-keyword task. A customized DS-CNN was therefore developed while keeping the architecture lightweight.

The principal changes were:

- redesign of the initial stem to extract stronger features from MFCC inputs;
- increase of the number of channels from 64 to 128;
- increase to five DS-CNN blocks;
- addition of two conventional 3×3 convolution layers before classification;
- adaptive global average pooling to obtain a fixed-size representation;
- dropout with probability 0.2 before the final classifier; and
- a final head consisting of global pooling, dropout, and a fully connected layer mapping 64 features to 21 output classes.

The 21 output classes correspond to the 20 keywords plus the unknown class.

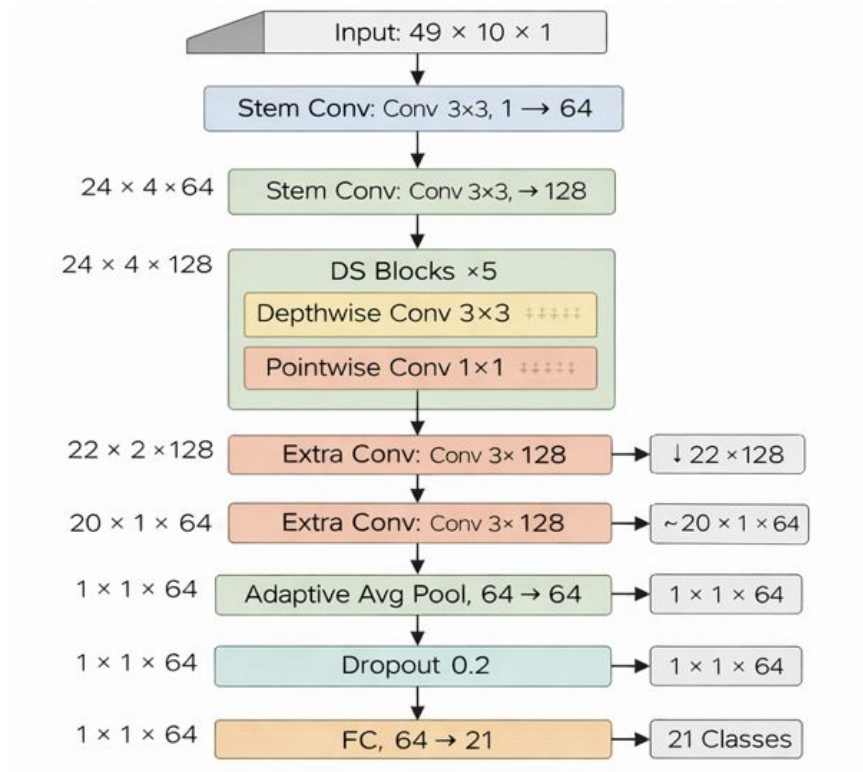


Figure 4.2: Customized DS-CNN architecture for GAP9.

4.3 GAP9 Results

The baseline DS-CNN contains 94,366 parameters, while the customized network contains 387,840 parameters. Before QAT, the corresponding accuracies were 88.84% and 91.26%. After QAT, the accuracies increased to 90.78% and 92.50%, respectively.

Table 4.1: GAP9 model comparison.

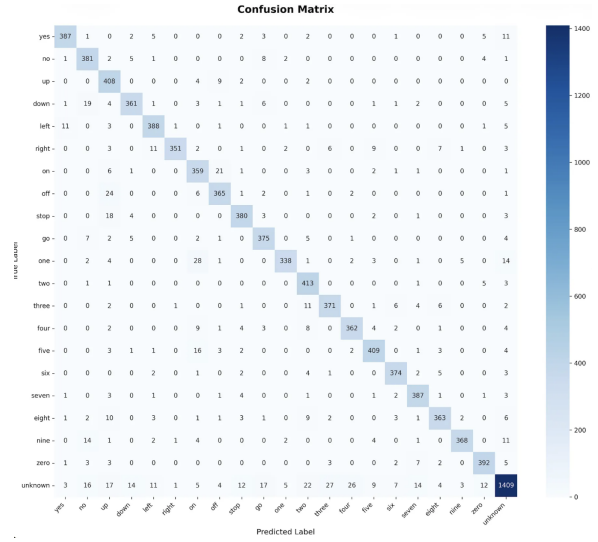
Metric	Baseline DS-CNN	Customized DS-CNN
Parameters	94,366	387,840
Accuracy before QAT	88.84%	91.26%
Accuracy after QAT	90.78%	92.50%
Model size	~ 379 KB	~ 1528 KB
INT8 model size	~ 95 KB	~ 383 KB
MACs	2,078,848	37,662,144
MAC/cycle	2.28269	6.42918
Inference time, without NE16	~ 24.6 ms	~ 15.8 ms
MAC/cycle, with NE16	7.2601	20.3917
Inference time, with NE16	0.774 ms	~ 4.99 ms

PER-CLASS ACCURACY:

yes	(Class 0): 92.36% (387/ 419)
no	(Class 1): 94.07% (381/ 405)
up	(Class 2): 96.00% (408/ 425)
down	(Class 3): 88.92% (361/ 406)
left	(Class 4): 94.17% (388/ 412)
right	(Class 5): 88.64% (351/ 396)
on	(Class 6): 90.66% (359/ 396)
off	(Class 7): 90.80% (365/ 402)
stop	(Class 8): 92.46% (380/ 411)
go	(Class 9): 93.28% (375/ 402)
one	(Class 10): 84.71% (338/ 399)
two	(Class 11): 97.41% (413/ 424)
three	(Class 12): 91.60% (371/ 405)
four	(Class 13): 90.50% (362/ 400)
five	(Class 14): 91.91% (409/ 445)
six	(Class 15): 94.92% (374/ 394)
seven	(Class 16): 95.32% (387/ 406)
eight	(Class 17): 88.97% (363/ 408)
nine	(Class 18): 90.20% (368/ 408)
zero	(Class 19): 93.78% (392/ 418)
unknown	(Class 20): 86.02% (1409/1638)

Mean: 91.75% $\pm$ 3.13%

(a) Baseline per-class accuracy.



(b) Baseline confusion matrix.

Figure 4.3: Baseline DS-CNN evaluation results on GAP9.

Final performance:

- num cycles: 910700
- MACs: 2078848
- MAC/cycle: 2.28269
- n. of Cores: 8

(a) Baseline performance summary.

Final performance:

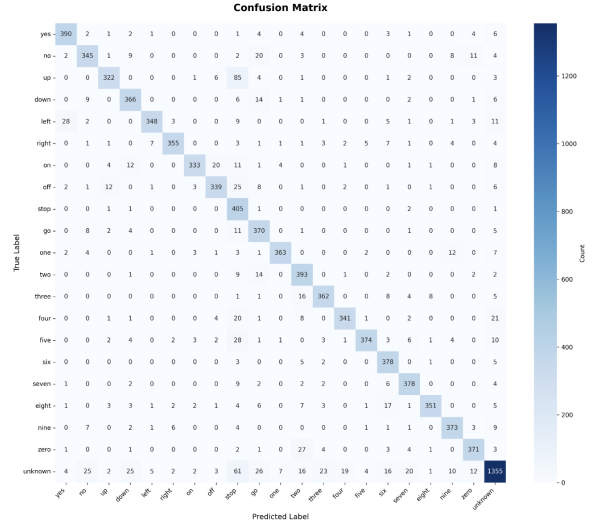
- num cycles: 5858003
- MACs: 37662144
- MAC/cycle: 6.42918
- n. of Cores: 8

(b) Customized model performance summary.

Figure 4.4: GAP9 performance summaries reported by the implementation.

PER-CLASS ACCURACY:		
yes	(Class 0):	93.08% (390/ 419)
no	(Class 1):	85.19% (345/ 405)
up	(Class 2):	75.76% (322/ 425)
down	(Class 3):	90.15% (366/ 406)
left	(Class 4):	84.47% (348/ 412)
right	(Class 5):	89.65% (355/ 396)
on	(Class 6):	84.09% (333/ 396)
off	(Class 7):	84.33% (339/ 402)
stop	(Class 8):	98.54% (405/ 411)
go	(Class 9):	92.04% (370/ 402)
one	(Class 10):	90.98% (363/ 399)
two	(Class 11):	92.69% (393/ 424)
three	(Class 12):	89.38% (362/ 405)
four	(Class 13):	85.25% (341/ 400)
five	(Class 14):	84.04% (374/ 445)
six	(Class 15):	95.94% (378/ 394)
seven	(Class 16):	93.10% (378/ 406)
eight	(Class 17):	86.03% (351/ 408)
nine	(Class 18):	91.42% (373/ 408)
zero	(Class 19):	88.76% (371/ 418)
unknown	(Class 20):	82.72% (1355/1638)
Mean: 88.46% $\pm$ 5.09%		

(a) Customized per-class accuracy.



(b) Customized confusion matrix.

Figure 4.5: Customized DS-CNN evaluation results on GAP9.

4.4 Discussion

The customized model improves post-QAT accuracy by 1.72 percentage points over the baseline, from 90.78% to 92.50%. This improvement comes with a substantially larger parameter count and memory footprint. At the same time, the reported NE16 inference time of the customized model is approximately 4.99 ms, compared with 0.774 ms for the baseline.

The results illustrate an important embedded-AI trade-off: additional representational capacity can improve recognition accuracy, but it can also increase memory requirements and accelerator workload. The baseline model therefore remains attractive when latency and compactness are prioritized, while the customized model is preferable when accuracy is the primary objective.

Chapter 5

Keyword Spotting on MAX78000FTHR

5.1 Experimental Strategy

The MAX78000FTHR experiments explored several model-development strategies for KWS20. The project considered:

1. a small baseline KWS20 model;
2. a larger teacher model followed by knowledge distillation into a smaller student;
3. a neural architecture search model following the general direction of a previously reported raw-audio KWS approach.

The referenced prior work used folded raw audio and Once-For-All neural architecture search for a KWS12 system and reported accuracy above 96%. In the present project, the task was instead KWS20.

5.2 Teacher Model

The teacher model uses a two-stage hybrid architecture. A one-dimensional CNN stage with four blocks extracts temporal features from the raw audio. The representation is then reshaped for a two-dimensional CNN stage containing four blocks that learn spatial relationships. The output consists of global pooling, dropout, and a fully connected layer producing 21-class predictions.

5.3 Knowledge Distillation

The student model was designed to be considerably lighter than the teacher while retaining useful information learned by the larger network.

The student contains a one-dimensional CNN stage with kernel size 1 and channel progression $128 \rightarrow 100 \rightarrow 100 \rightarrow 50 \rightarrow 16$. The resulting tensor is reshaped from $(B, 16, 128)$ to $(B, 16, 16, 8)$ and processed by a two-dimensional CNN stage with kernel size 3. The 2D stage uses channel progression $16 \rightarrow 32 \rightarrow 64 \rightarrow 64 \rightarrow 30 \rightarrow 7$.

The final representation is flattened from 7×128 features and mapped to 21 output classes.

```
=====
MODEL SUMMARY
=====
Total parameters: 445,941
Trainable parameters: 445,941
Non-trainable parameters: 0
Model size: 1747.77 KB (1.71 MB)
```

Figure 5.1: Teacher-model training and model-summary information reported in the project.

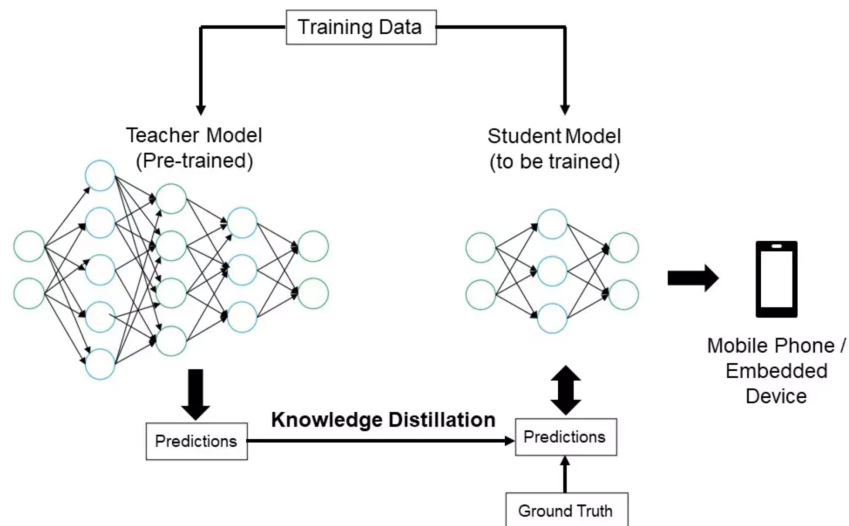


Figure 5.2: Knowledge-distillation training concept represented in the project material.

5.3.1 Distillation Objective

The student is trained using both the ground-truth class label and the probability distribution produced by the teacher. The temperature-scaled probabilities are

$$p_T = \text{softmax}\left(\frac{z_T}{T}\right), \quad p_S = \text{softmax}\left(\frac{z_S}{T}\right), \quad (5.1)$$

where z_T and z_S are the teacher and student logits, respectively, and the temperature used in the project is $T = 4.0$.

The combined loss is

$$L = \alpha \text{CE}(y, z_S) + (1 - \alpha)T^2 \text{KL}(p_T \| p_S), \quad (5.2)$$

with

$$\alpha = 0.7, \quad 1 - \alpha = 0.3.$$

The cross-entropy term trains the student against the correct class label, while the KL-divergence term encourages the student to reproduce the teacher’s output distribution.

5.4 Knowledge-Distilled Student Results

The final student model contains 126,580 parameters. Its accuracy improved from 77.86% before knowledge distillation to 86.63% after distillation, and then to 89.084% after QAT.

Table 5.1: Knowledge-distilled student model on MAX78000FTHR.

Metric	Value
Parameters	126,580
Accuracy before knowledge distillation	77.86%
Accuracy after knowledge distillation	86.63%
Accuracy after QAT	89.084%
Model size	~ 123 KB
MACC	13.8 M
Inference time	2.013 ms

5.5 Energy and Throughput Estimates

The project reports an inference time of 2.013 ms and an estimated active power of 65.87 mW at an estimated voltage of 3.7 V. The reported energy per inference is calculated as

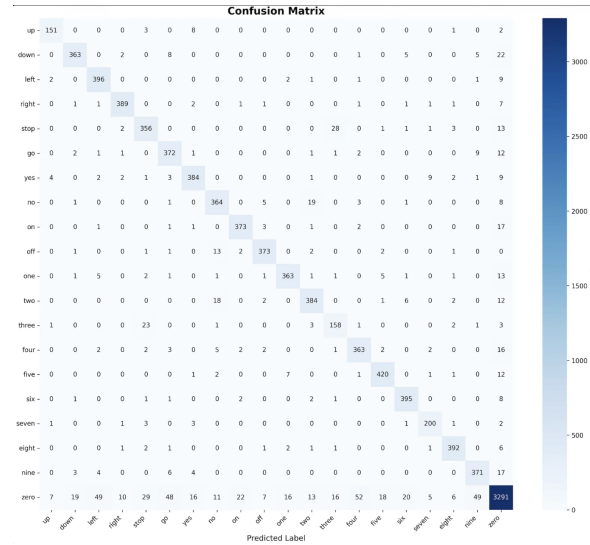
$$E = P t = 0.06587 \text{ W} \times 0.002013 \text{ s} \approx 0.0001326 \text{ J} = 132.6 \mu\text{J}. \quad (5.3)$$

For 13,801,088 MACC operations, the corresponding energy per MACC is

$$\frac{132.6 \mu\text{J}}{13,801,088} \approx 0.00961 \text{ nJ/MACC}. \quad (5.4)$$

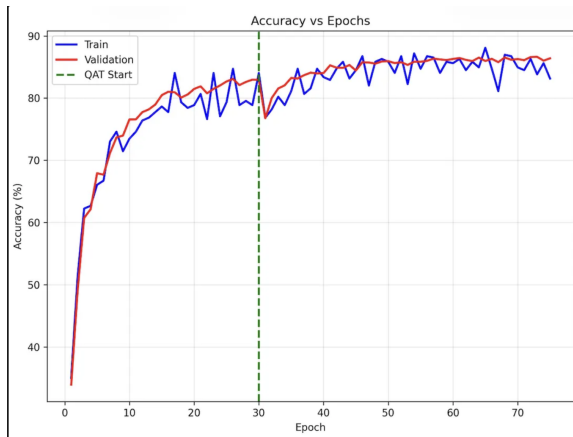
	precision	recall	f1-score	support
up	0.8908	0.9394	0.9145	165
down	0.9036	0.8547	0.8785	406
left	0.8134	0.9102	0.8591	412
right	0.9189	0.9483	0.9333	406
stop	0.8627	0.8691	0.8659	405
go	0.7756	0.8682	0.8192	402
yes	0.9272	0.9139	0.9205	418
no	0.8442	0.8085	0.8259	402
on	0.9173	0.8897	0.9033	399
off	0.8789	0.9167	0.8974	396
one	0.9263	0.8889	0.9072	396
two	0.8079	0.9106	0.8562	425
three	0.8743	0.7927	0.8315	193
four	0.8222	0.8900	0.8547	400
five	0.9213	0.9213	0.9213	445
six	0.9322	0.9367	0.9345	411
seven	0.9296	0.9340	0.9318	212
eight	0.8951	0.9412	0.9176	408
nine	0.8329	0.8741	0.8530	405
zero	0.0000	0.0000	0.0000	0
unknown	0.9257	0.8809	0.9028	3704
accuracy			0.8908	10810
macro avg	0.8381	0.8519	0.8442	10810
weighted avg	0.8930	0.8908	0.8912	10810

(a) Classification report.

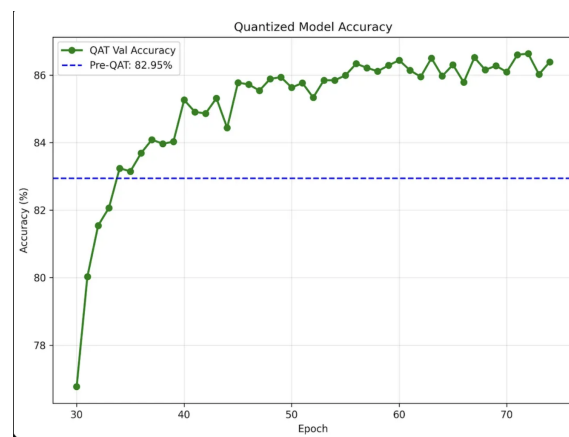


(b) Confusion matrix.

Figure 5.3: Evaluation results for the knowledge-distilled student.



(a) Training and validation accuracy.



(b) Quantized-model accuracy.

Figure 5.4: Training and QAT behavior of the MAX78000 student model.

The corresponding computational throughput is approximately

$$\frac{13,801,088}{0.002013} \approx 6.86 \text{ GMACC/s}, \quad (5.5)$$

and the estimated inference rate is

$$\frac{1}{0.002013} \approx 496.77 \text{ inferences/s}. \quad (5.6)$$

These energy and power values are estimates based on the active-power reference reported in the presentation and should therefore be interpreted as project-level estimates rather than a direct board-level power measurement.

5.6 Neural Architecture Search Model

A separate NAS model was also developed. Its 1D CNN stage uses 14 fused convolutional blocks with mixed kernel sizes of 1, 3, and 5, together with intermittent max-pooling and convolution operations. The main channel progression is

$128 \rightarrow 128 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 64 \rightarrow 128 \rightarrow 64 \rightarrow 128$.

The output is flattened to 512 features and passed to a fully connected layer for 21-class classification.

Table 5.2: NAS model performance reported for MAX78000FTHR.

Metric	Value
Parameters	426,076
Accuracy	88.76%
Model size	~ 1675 KB

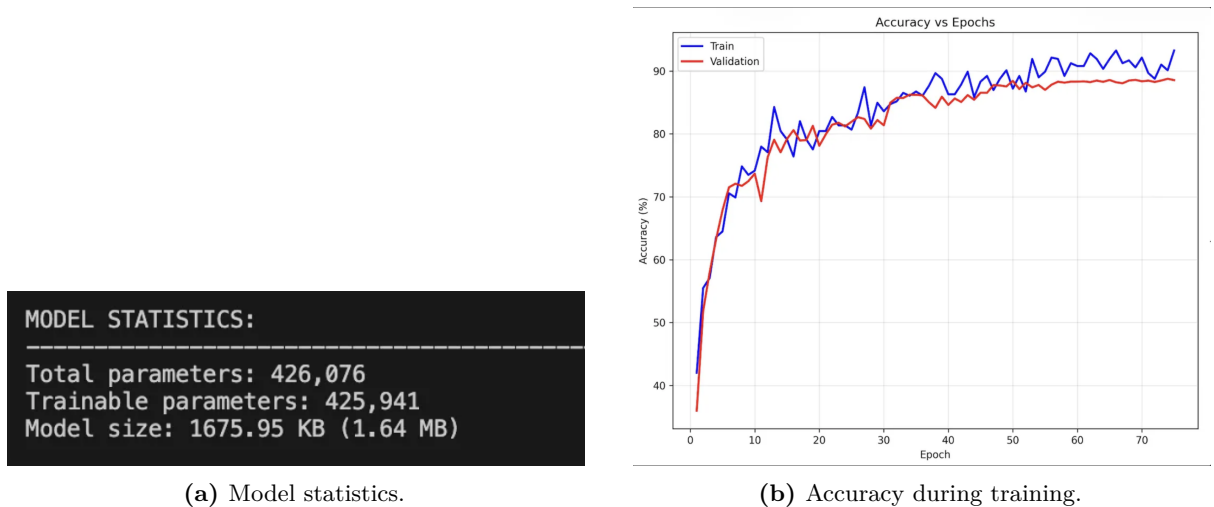
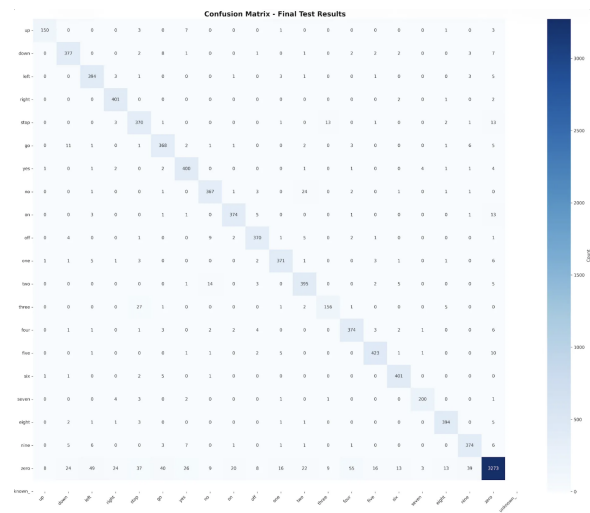


Figure 5.5: NAS model training and model statistics.

CLASSIFICATION REPORT				
	precision	recall	f1-score	support
up	0.9317	0.9091	0.9202	165
down	0.8850	0.9286	0.9062	406
left	0.8510	0.9563	0.9006	412
right	0.9134	0.9877	0.9491	406
stop	0.8150	0.9136	0.8615	405
go	0.8499	0.9154	0.8814	402
yes	0.8929	0.9569	0.9238	418
no	0.9084	0.9129	0.9107	402
on	0.9303	0.9373	0.9338	399
off	0.9296	0.9343	0.9320	396
one	0.9229	0.9369	0.9298	396
two	0.8662	0.9294	0.8967	425
three	0.8715	0.8083	0.8387	193
four	0.8462	0.9350	0.8884	400
five	0.9358	0.9506	0.9431	445
six	0.9369	0.9757	0.9559	411
seven	0.9569	0.9434	0.9501	212
eight	0.9381	0.9657	0.9517	408
nine	0.8718	0.9235	0.8969	405
zero	0.0000	0.0000	0.0000	0
unknown	0.9727	0.8836	0.9260	3704
accuracy			0.9188	10810
macro avg	0.8584	0.8859	0.8713	10810
weighted avg	0.9220	0.9188	0.9191	10810

(a) Classification report.



(b) Confusion matrix.

Figure 5.6: Evaluation results for the NAS model.

Chapter 6

Comparative Analysis

6.1 Overall Model Comparison

The final comparison reported in the project is summarized in table 6.1. The models represent different points in the accuracy–complexity–latency design space.

Table 6.1: Overall comparison of the reported KWS models.

Model	Platform	Params	Acc.	Size	INT8	MACs/MACC	Time
Baseline DS-CNN	GAP9	94,366	90.78%	~ 379 KB	~ 95 KB	2 M	0.774 ms
Customized DS-CNN	GAP9	387,840	92.50%	~ 1528 KB	~ 383 KB	37 M	4.99 ms
KD student	MAX78000	126,580	89.084%	~ 123 KB	~ 123 KB	13.8 M	2.013 ms
NAS	MAX78000	426,076	88.76%	~ 1675 KB	–	–	–

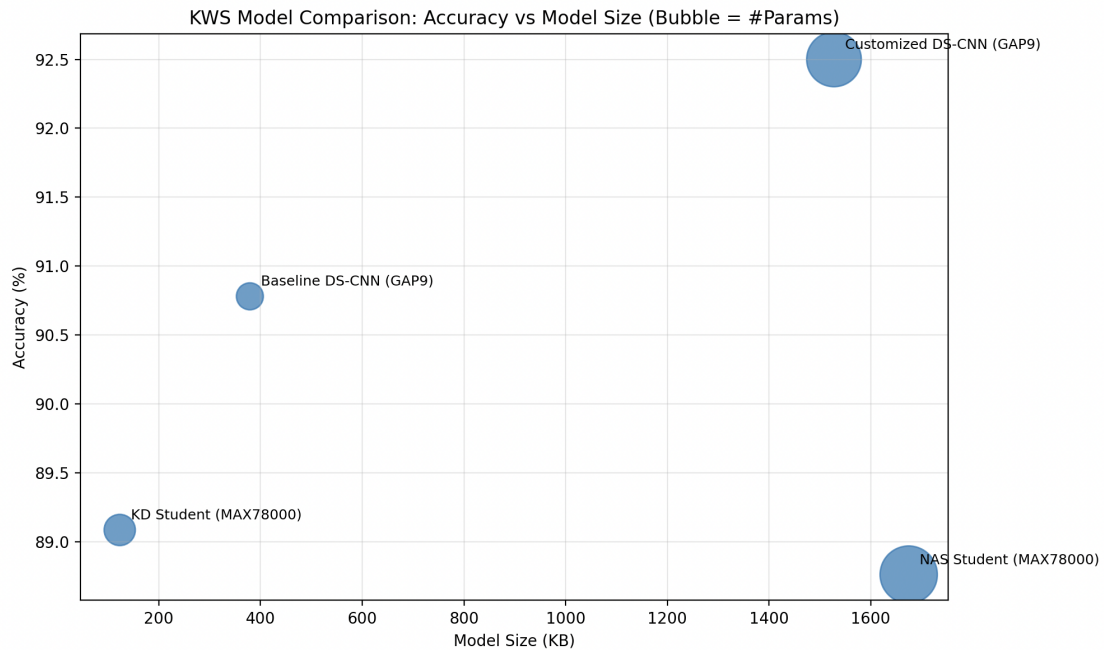


Figure 6.1: Overall model comparison reproduced from the project evaluation.

6.2 Accuracy, Size, and Latency Trade-offs

The customized GAP9 DS-CNN obtains the highest reported accuracy at 92.50%, but it also has the largest memory footprint among the compact deployed models and requires approximately 4.99 ms per inference with NE16. The baseline GAP9 DS-CNN provides the shortest reported inference time, at 0.774 ms, while retaining 90.78% accuracy after QAT.

The MAX78000 knowledge-distilled student provides a different balance. With 126,580 parameters and a model size of approximately 123 KB, it is substantially smaller than the GAP9 baseline and customized models. Its 89.084% post-QAT accuracy is lower than both GAP9 models, but its measured 2.013 ms inference time remains in the millisecond range while the model occupies a small memory footprint.

The NAS model does not provide an advantage over the knowledge-distilled student in the reported results: it has more than three times as many parameters, a substantially larger model size, and a slightly lower accuracy of 88.76%. The project therefore selects the knowledge-distilled student as the best overall deployed model.

6.3 Selected Deployment Model

According to the project conclusion, the KD student is the preferred deployed model because it combines:

- 89.08% accuracy after QAT,
- approximately 123 KB model size,
- 126,580 parameters,
- 13.8 M MACC operations, and
- verified 2.013 ms inference time on MAX78000FTHR.

The project notes that this model is approximately four times smaller than the GAP9 baseline model while providing comparable accuracy. The comparison should be understood in the context of the two different hardware platforms and their respective inference implementations.

Chapter 7

Conclusion

This project investigated low-power keyword spotting for a 20-keyword task on GAP9 and MAX78000FTHR. The work demonstrated two complementary approaches to embedded KWS optimization.

On GAP9, the customized DS-CNN improved accuracy from 90.78% for the baseline after QAT to 92.50%, but this improvement required a larger model and significantly more MAC operations. The baseline model achieved a particularly short reported NE16 inference time of 0.774 ms.

On MAX78000FTHR, knowledge distillation provided a practical method for transferring information from a larger hybrid teacher to a compact student. The student improved from 77.86% before distillation to 86.63% after distillation and reached 89.084% after QAT. Its approximately 123 KB model size and 2.013 ms measured inference time make it attractive for resource-constrained deployment.

The NAS model reached 88.76% accuracy with 426,076 parameters and an approximately 1675 KB model size. Based on the reported results, it therefore did not outperform the knowledge-distilled student in the overall deployment trade-off.

Overall, the project highlights that the best embedded KWS model is not necessarily the model with the highest classification accuracy. Model size, accelerator execution, latency, and energy efficiency are equally important when the target application requires always-on operation.

Chapter 8

Future Work

The project identifies two main directions for future work.

8.1 Knowledge Distillation on GAP9

Knowledge distillation could be applied to GAP9 to investigate whether the accuracy of a larger model can be transferred into a smaller DS-CNN while reducing memory and computational cost. This would provide a direct comparison between architectural scaling and distillation-based compression on the same platform.

8.2 Improved Distillation Strategies

The KD student could be further optimized by tuning the temperature T and the relative weighting between the cross-entropy and KL-divergence terms. More advanced distillation methods could also be evaluated, including intermediate feature matching, in which the student is trained to reproduce internal representations of the teacher in addition to its final probability distribution.

References

- [1] Özdemir et al., “An ultra low power keyword spotting system using raw audio and neural architecture search on MAX78000,” arXiv, 2021. Cited in the project presentation as the reference for the raw-audio and neural-architecture-search approach.
- [2] Analog Devices, *MAX78000FTHR Datasheet*, 2023. Hardware reference cited in the project presentation.
- [3] M. Fariselli, *GAP9 Architecture*, EFCL Seminar, ETH Zurich. Hardware architecture reference cited in the project presentation.
- [4] Analytics Vidhya, “Knowledge Distillation: Theory and End-to-End Case Study,” January 2022. Background source cited in the project presentation for knowledge distillation.
- [5] S. Jang, F. Kawsar, and C. Min, “Thermal Characterization of AI Applications on AI Accelerators-equipped Microcontrollers,” 2024. Reference cited in the project presentation for the active-power estimate used in the energy calculation.

Source and Scope Note

This report is a research-style reorganization of the supplied project presentation. Numerical values, model configurations, experiment results, and references are reproduced from the presentation. No additional experimental results have been introduced. Where the presentation provides estimates rather than direct measurements (for example, active power and energy per inference), the report labels them as estimates.